Neo4j

For Banking

With a live demo

Press **S** for speaker notes · **Esc** for overview · **F** for fullscreen

Agenda

1. **Where we are now** — the industry reality
2. **Pain of the status quo** — where traditional databases do not keep up
3. **Paradigm shift** — from tables to graphs
4. **Solution architecture** — how Neo4j is built
5. **Demo** — fraud, from questions to answers
6. **Customer success** — proof of value
7. **Call to action** — next steps

1 · Where we are now

The industry reality

The world got faster (so did the fraud)

- **Digital transactions and instant payments** have exploded in volume.
- **Financial crime has grown in tandem** — greater volume, greater complexity, more sophisticated.
- Fraud rarely acts alone: it **hides inside highly connected networks** — shared devices, mule accounts, synthetic identities, collusion rings.

> *Money can move instantly now, fraud detection that runs overnight is already too slow.*

2 · Pain of the Status Quo

Where traditional databases do not keep up

- Traditional Relational Databases (RDBMS) are built to tabulate data, not connect it.
- A 5/6-hop SQL JOIN query for tracing a money laundering network takes hours, resources and hard to write.
- Not tackling this has real cost: **compliance exposure**, **delayed transaction blocks**, and slower response to active incidents.

3 · The Paradigm Shift

From tables to graphs

- Move from **rows and columns** to **Nodes** and **Relationships** — a model built for how things connect, not just what they are.
- **To catch connected criminals, you need connected data.**

```
(Customer)-[:HAS_CARD]->(Card)-[:FUNDS]->(Purchase)-[:PAID_TO]->(Merchant)
```

Neo4j stores exactly this so there's no need to compute this relation for every query, speeding up traversals.

4 · Solution Architecture

Complex SQL vs. one readable path

Find merchants paid by a specific customer's card:

SQL

```sql
SELECT DISTINCT m.name
FROM customer c
JOIN card cd    ON cd.cif = c.cif
JOIN purchase p ON p.card_number = cd.card_number
JOIN merchant m ON m.name = p.merchant
WHERE c.cif = '5';
```

Cypher

```
MATCH (c:Customer {cif:'5'})-[:HAS_CARD]->(
      -[:FUNDS]->(:Purchase)-[:PAID_TO]->(m
RETURN DISTINCT m.name;
```

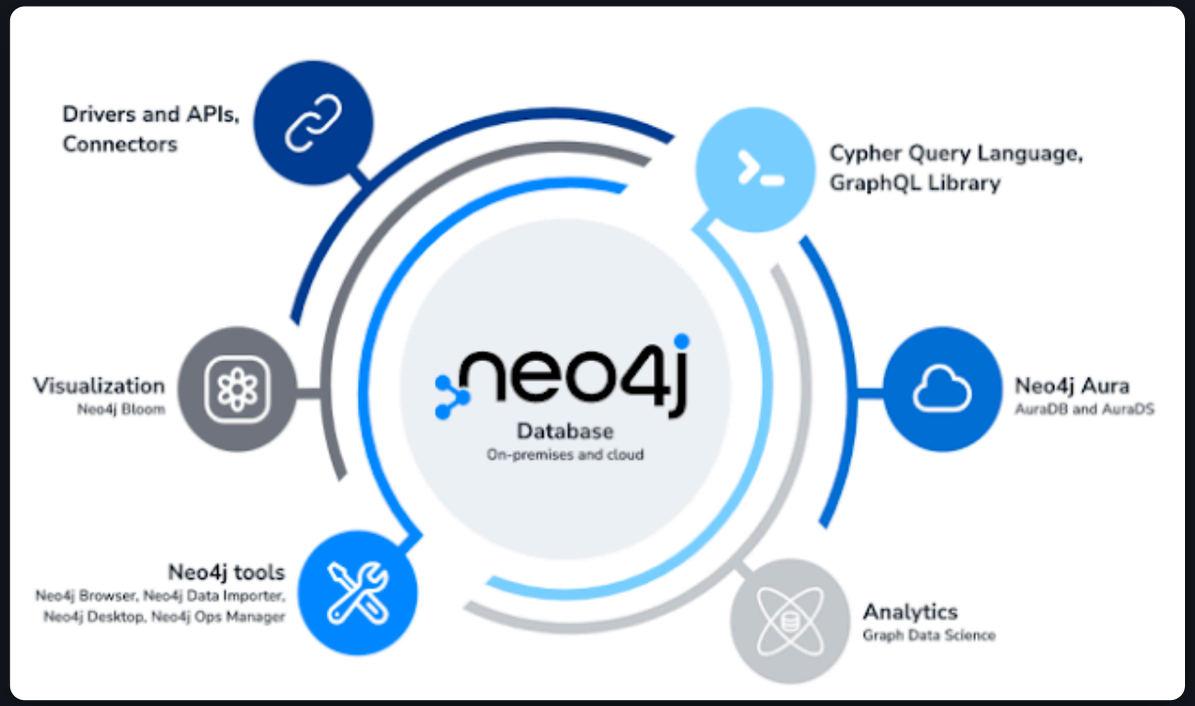
- In Neo4j, **relationships are first-class citizens** — stored on disk, not recomputed at query time.
- **Add new node types without downtime** or breaking a rigid schema — the model grows additively as the business changes.

Recursive questions like "anyone within 4 transfer hops of an account" are just `-[:SENT_TO|RECEIVED_BY*1..4]-`.

Built for traversal

- **Index-free adjacency** — each node holds direct pointers to its neighbours, so traversing a relationship is a pointer-chase, not an index lookup. Query cost stays **flat per hop** instead of compounding.
- **ACID compliance** — atomicity, consistency, isolation, durability - full transactional integrity even in power failures.
- **Cypher** — a human-readable, intuitive, pattern-based query language.

The Neo4j ecosystem



Neo4j platform: database, data science, visualization and AI/GraphRAG tooling in one.

Graph Data Science: topology-aware fraud detection

- Special-purpose **graph algorithms** run directly where the data lives — no export to a separate analytics stack.
- **Louvain community detection** groups accounts by how tightly they're connected, surfacing collusion rings that per-transaction rules never see.
- and many other graph algorithms around centrality, community detection, similarity, path finding, etc.

```
// Community detection groups colluding accounts into rings
CALL gds.louvain.stream('xfers')
YIELD nodeId, communityId;
```

Banking-grade compliance & security

- **RBAC and fine-grained access control** — row/property-level permissions for regulated data.
- **Support for GDPR** and similar regulatory regimes — data residency, right-to-erasure, auditability.
- **Encryption at rest and in transit**, meeting the same bar as the rest of the banking stack.
- SOC Type 2: https://neo4j.com/blog/news/neo4j-is-now-soc2-type-ii-compliant/

Other use cases

Neo4j excels at **connected** data - which happens more than one expects:

- employees
- network and security
- suppliers
- process
- product
- customers
- transactions

5 · Demo — Fraud

What we'll walk through

1. **How an analyst investigates and uncovers fraud** — including **ID collisions**.
2. **Converting tabular data to a graph**.
3. **Writing a graph query that surfaces a business insight**.
4. **Running topology-aware Graph Data Science algorithms** to detect fraud rings directly in the database.

6 · Customer Success

BNP Paribas Personal Finance

Processing **800,000+ credit applications a year**, facing fraud rings that reuse and subtly modify personal details across applications.

With Neo4j:

- **20% reduction in fraud**, while maintaining approval volumes
- **2-second maximum query latency** for real-time scoring decisions
- **Millisecond-level comparisons** against historical application data

> *"With Neo4j, we have a much better view of each consumer's application. The expansive data context enables us to discern intricate patterns, even uncovering links to known fraudsters."*

neo4j.com/customer-stories/bnp-paribas-personal-finance

7 · Call to Action

Next steps

Graph enablement session for the data engineering team

- Bring your own tables — walk through mapping existing schemas (customers, cards, transfers, merchants) to a native graph model.
- Hands-on with Cypher and GDS against a working environment.
- Leave with a scoped, high-value question your team can stand up first.

The connections in your data are an asset.